2520030117

Practical 11 — POSIX Threads and Mutex

AIM

To develop a multithreaded counter application using POSIX threads (`pthread_create()` and `pthread_join()`), demonstrate a race condition by allowing multiple threads to update a shared variable concurrently, and eliminate the race condition using mutex locks. The outputs are compared to analyze the effect of synchronization.

OBJECTIVES

1. Create multiple threads using `pthread_create()`.
2. Use `pthread_join()` to wait for thread completion.
3. Demonstrate a **race condition** on a shared counter.
4. Use `pthread_mutex_lock()` and `pthread_mutex_unlock()` to protect the shared variable.
5. Compare synchronized and unsynchronized outputs.
6. Analyze the impact of synchronization on the final counter value.

REQUIREMENTS

- Ubuntu/Linux
- GCC compiler
- POSIX Threads (`pthread`)

THEORY

POSIX Threads (Pthreads):
POSIX threads allow multiple threads to execute concurrently within the same process.

`pthread_create()`
Creates a new thread and starts its execution.

`pthread_join()`
Makes the main thread wait until the specified thread finishes.

Race Condition:
A race condition occurs when multiple threads access and modify the same shared variable at the same time. Because the operations can overlap, the final result may be incorrect or unpredictable.

Mutex:
A mutex (mutual exclusion lock) allows only one thread at a time to access the critical section.

Functions used:

pthread_mutex_lock()
pthread_mutex_unlock()

ALGORITHM — WITHOUT MUTEX

1. Start the program.
2. Declare a shared counter variable.
3. Create multiple threads using `pthread_create()`.
4. Each thread increments the shared counter repeatedly.
5. Allow all threads to access the counter concurrently.
6. Wait for all threads using `pthread_join()`.
7. Display the final counter value.
8. Observe the race condition.
9. Stop.

ALGORITHM — WITH MUTEX

1. Start the program.
2. Declare a shared counter variable.
3. Initialize a mutex.
4. Create multiple threads.
5. Each thread locks the mutex before modifying the counter.
6. Increment the shared counter.
7. Unlock the mutex after the update.
8. Wait for all threads using `pthread_join()`.
9. Display the final counter value.
10. Destroy the mutex and stop.

EXPECTED COMPARISON

Version	Synchronization	Result
Without mutex	❌ No	May produce an incorrect/unpredictable counter due to race condition
With mutex	✅ Yes	Correct and consistent counter value

RESULT

Thus, a multithreaded counter application was successfully implemented using POSIX threads. The race condition caused by concurrent access to a shared variable was demonstrated, and synchronization using mutex locks was implemented successfully. The mutex-protected version provides controlled access to the shared counter and produces the expected result.

Final comparison for your record

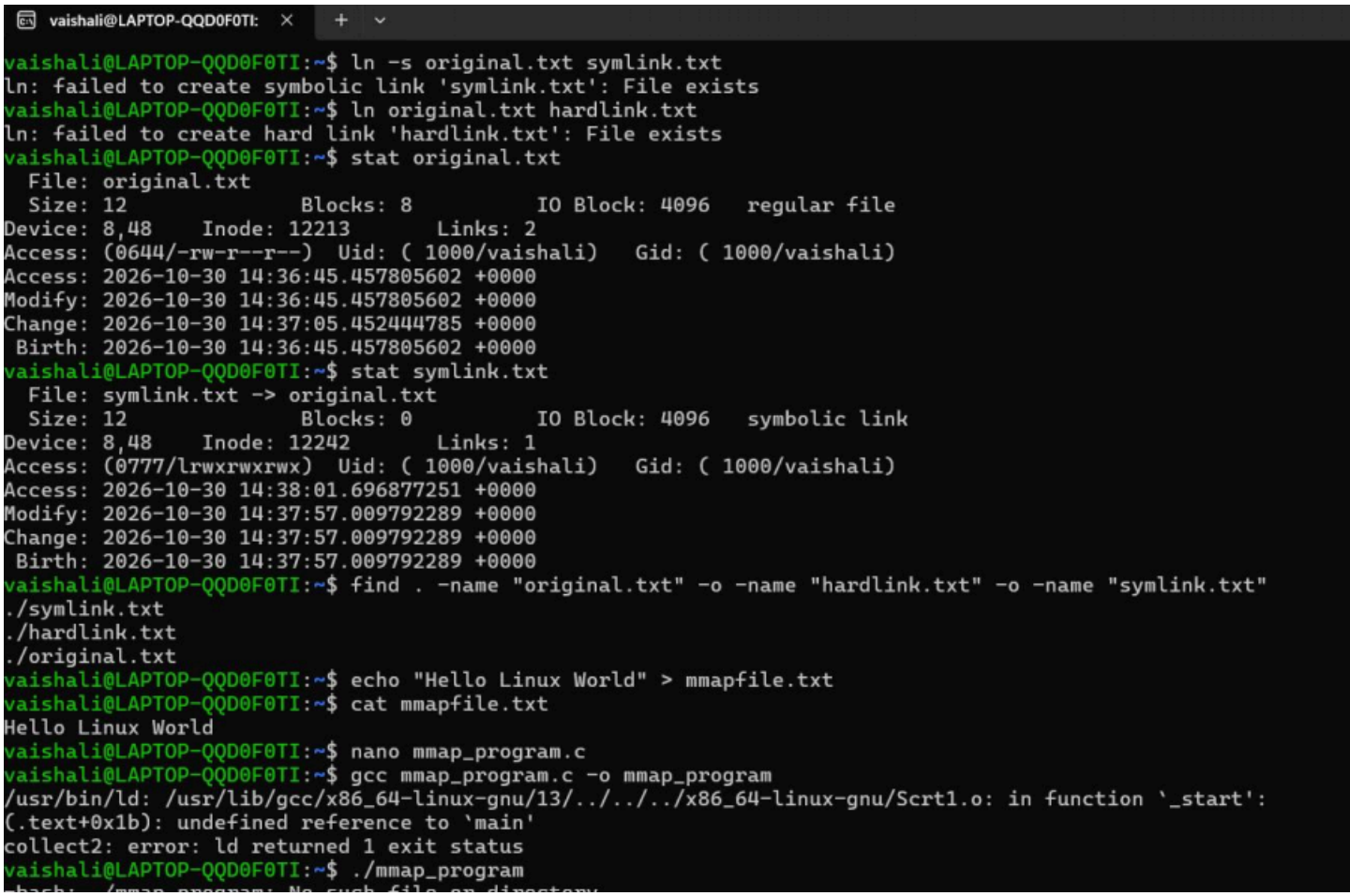
Version Mutex Final Counter

Race condition No Varies

Synchronized Yes 4,000,000

RESULT

Thus, a multithreaded counter application was successfully implemented using POSIX threads. The race condition caused by concurrent access to a shared variable was demonstrated. Mutex synchronization using `pthread_mutex_lock()` and `pthread_mutex_unlock()` was then implemented, producing the correct final counter value of 4,000,000.



```
vaishali@LAPTOP-QQD0F0TI:~$ cd /mnt/c/Users/Vaishali/OneDrive/Desktop/OS/Practical/P
-bash: cd: /mnt/c/Users/Vaishali/OneDrive/Desktop/OS/Practical/Practical-10: No such
vaishali@LAPTOP-QQD0F0TI:~$ echo "Hello Linux" > original.txt
vaishali@LAPTOP-QQD0F0TI:~$ ls -l original.txt
-rw-r--r-- 1 vaishali vaishali 12 Oct 30  2026 original.txt
vaishali@LAPTOP-QQD0F0TI:~$ stat original.txt
  File: original.txt
  Size: 12              Blocks: 8          IO Block: 4096   regular file
Device: 8,48    Inode: 12213       Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/vaishali)   Gid: ( 1000/vaishali)
Access: 2026-10-30 14:36:45.457805602 +0000
Modify: 2026-10-30 14:36:45.457805602 +0000
Change: 2026-10-30 14:36:45.457805602 +0000
 Birth: 2026-10-30 14:36:45.457805602 +0000
vaishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink.txt
vaishali@LAPTOP-QQD0F0TI:~$ ls -i original.txt hardlink.txt
12213 hardlink.txt  12213 original.txt
vaishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink.txt
ln: failed to create hard link 'hardlink.txt': File exists
vaishali@LAPTOP-QQD0F0TI:~$ ls -i original.txt hardlink.txt
12213 hardlink.txt  12213 original.txt
vaishali@LAPTOP-QQD0F0TI:~$
```

```
ErrorLogger.class       memory_allocation.c                              race
InputValidator.class    os-operating-system-and-system-programming-OSSP  race.c
OSSP                    os_practical5                                    shell
Prime-ossp-os           os_practical7                                    shell.c
Skill21.class           p10                                              shell.c.save
Skill21.java            p11                                              shell.c.save.1
UserService.class       p9                                               skill22
copy_on_write           pipeline_ipc                                     skill24
copy_on_write.c         process.c.save                                   student.txt
directory_name          program                                          text.txt
```